

DEMO DAY BUILD GUIDE

Kronos Copilot

Local market forecasting for NSE and BSE, with optional low-cost AI explanation

Project status: working local prototype | Prepared: 20 August 2026

This guide documents the implemented system, the design choices behind it, and the path to a reliable Demo Day presentation.

1. Executive Summary

Kronos Copilot is a local financial-forecasting dashboard designed for a Demo Day presentation. It accepts a CSV market file or a live Indian-market ticker, runs Kronos-base on this laptop, and presents a short forecast in a simple dashboard.

The system keeps the expensive or sensitive parts deliberately small. The forecast runs locally. The OpenAI API is optional and is called only when the user asks for an explanation of an already-created compact JSON summary. The API key stays on the local server and is not sent to the browser.

Current verified result: ITC.NS live 5-minute data supplied 400 rows. Kronos-base forecast 24 future intervals, from a last observed close of 270.8 to a final forecast close of 269.9708 (-0.306% down).

2. Research and Scope

The project is intentionally a forecasting copilot, not a trading-advice product. Its purpose is to demonstrate a clean AI system: a specialist financial model produces a numerical estimate, and a language model explains only the final compact result in plain English.

Why Kronos

- Kronos is an open-source K-line financial foundation model maintained in the official shiyu-coder/Kronos repository.
- Its model zoo lists Kronos-base at roughly 102.3 million parameters, with a 512-context window. This is the strongest practical open-source choice for the local laptop workflow.
- The project uses 400 historical OHLCVA rows and forecasts 24 future rows, staying comfortably within the model context while retaining useful recent intraday history.

Why NSE and BSE first

The target audience is Indian-market users. The dashboard therefore defaults to NSE and accepts BSE selection. A simple entry such as RELIANCE becomes RELIANCE.NS for NSE or RELIANCE.BO for BSE before data is requested. Explicit .NS, .BO, and index symbols are preserved.

Data caveat: The prototype uses yfinance for live-demo data. It is convenient for a demo but may be delayed, rate-limited, or unavailable. It must not be represented as an official real-time exchange feed.

3. Architecture

The architecture separates numerical forecasting from natural-language explanation. This makes the app easier to explain, cheaper to run, and safer to demo.

Stage	What happens	Where
1. Market input	User uploads OHLCVA CSV or enters an NSE/BSE ticker. The app normalizes timestamps, open, high, low, close, volume, and amount.	Local server
2. Kronos forecast	The last 400 rows are passed to NeoQuasar/Kronos-base. It writes 24 predicted rows to forecast.csv.	Local CPU
3. Compact summary	The app converts forecast essentials into forecast_summary.json: price range, direction, percent change, and metadata.	Local server
4. Optional explanation	When the user presses Explain, gpt-5-mini receives compact JSON only. The response is cached against a summary fingerprint.	OpenAI API
5. Dashboard	The browser shows the summary, forecast direction, range, source, and optional explanation.	Local browser

Forecast data is never sent to OpenAI in raw form. The language-model prompt uses only the small JSON output, so it cannot consume the full price history by accident.

4. What Was Built

The following components exist in the working local prototype.

Component	Implementation	Reason
Kronos runtime	Official Kronos source is stored in vendor/Kronos-master and runs through the project Python environment.	Keeps model code local and reproducible.
Forecast script	src/first_forecast.py validates seven columns, retains 400 rows, forecasts 24 rows, and saves CSV plus JSON outputs.	A stable handoff for the dashboard and explanation layer.
Live India data	app/server.py uses yfinance five-minute history and maps NSE/BSE symbols to Yahoo suffixes.	Familiar Indian ticker names for the Demo Day flow.
Dashboard	app/dashboard.html provides ticker buttons, exchange selection, CSV input, summary cards, and an explanation action.	A simple visible workflow for judges and non-technical users.
API security	.env.local is ignored by Git and loaded only by the local Python server.	The browser never receives the OpenAI key.

5. Low-Cost OpenAI Design

OpenAI is not responsible for creating the forecast. Kronos does the numerical work locally. OpenAI is used only to make the result easier to understand after the user deliberately asks for it.

Decision	Cost and reliability effect
Use gpt-5-mini	A cost-sensitive model is sufficient for a short plain-language explanation; no complex research is requested.
Send compact JSON	The request contains summary fields, not 400 rows of OHLCVA data or a full chart.
Limit output	The endpoint requests three short bullets, at most 90 words, with max_output_tokens set to 140.
Use minimal reasoning	Avoids spending tokens on hidden elaboration or long prose for a simple explanation task.
Cache by forecast fingerprint	Repeated requests for exactly the same forecast return the saved explanation and make no additional API call.
Keep explanation optional	The forecast and dashboard still work even if no API key, quota, or network access is available.

Prompt boundary: The explanation instruction forbids recommendations and facts beyond the supplied JSON, and labels the result as an estimate rather than financial advice.

6. Verification Record

These checks were completed during the local build. They are useful evidence for the Demo Day explanation and a starting point for future automated tests.

Check	Result	What it proves
Kronos-base local inference	Passed	The selected open-source model loads and creates forecast.csv and forecast_summary.json on CPU.
CSV forecast endpoint	Passed	A dashboard upload reaches the server, validates the file, and triggers the forecast script.
OpenAI explanation endpoint	Passed	The server can read the local-only key, request a short explanation, and cache the answer.
NSE RELIANCE ticker	Passed	Entering RELIANCE with NSE resolves to RELIANCE.NS and produced a successful forecast during build verification.
BSE data lookup	Passed	The RELIANCE.BO data source returned sufficient recent intraday rows; BSE is available through the same suffix logic.

Current saved forecast

Metric	Saved value
Source	ITC.NS live 5-minute data
Model	NeoQuasar/Kronos-base
Observed close	270.8
Forecast final close	269.9708
Forecast range	269.9708 to 270.924
Projected change	-0.306% down

7. Demo Day Runbook

Use the local version for the presentation. It is the most reliable path because the forecast model, source code, and outputs stay on the laptop.

1. Start the local Python server from the project environment and open the dashboard at <http://127.0.0.1:8000/app/dashboard.html>.
2. Choose NSE, enter a familiar stock such as RELIANCE, TCS, HDFCBANK, or INFY, and run the live forecast. If live data is unavailable, use the included sample CSV as the fallback demonstration.
3. Point out that the system uses 400 recent rows to create 24 future intervals with Kronos-base running locally.
4. Show the compact structured forecast summary first. This makes it clear that the numerical estimate exists before the language model is called.
5. Press Explain once. Explain that the API receives only the small summary and then caches the result, so pressing it again does not spend another API request for the same forecast.
6. Close by stating the constraint: the product is a decision-support demo and not financial advice or an official real-time market-data terminal.

Before presenting: Open the dashboard once, verify a live ticker and the fallback CSV, confirm the API key is still present in `.env.local`, and keep the laptop connected to power and internet.

8. Deployment Path

The current implementation is deliberately local-first. Hosting should be a second phase after the Demo Day prototype is stable, because both the Kronos model and the secret API key need a trusted server environment.

Phase	Action	Acceptance check
1. Local release	Keep the local server on the laptop and package requirements, source, and fallback data flow.	A clean restart opens the dashboard and forecasts without code changes.
2. Server deployment	Deploy <code>app/server.py</code> to a Python service or VM. Store <code>OPENAI_API_KEY</code> in the host secret manager.	The server can fetch permitted data, run the model, and return results.
3. Model caching	Pre-cache the Hugging Face model and tokenizer in the deployment image or a persistent volume.	A restart avoids a slow first-download during the demo.
4. Performance upgrade	Use a suitable GPU host only if measured CPU response time is unsuitable for users.	Forecast response time meets the product goal.
5. Data hardening	Replace demo-oriented data access with a licensed NSE/BSE-compatible provider before production claims.	Data terms, latency, and coverage are documented.

Deployment guardrails

- Do not host the dashboard as a static page by itself: forecasting and API-key handling must remain on the server.
- Keep `.env.local` out of Git. In a hosted environment, use platform secrets instead of copying the local file.
- Add a request queue and per-user rate limit before opening public access, because model forecasting can be slower than an ordinary web request.
- Clearly label all outputs as estimates and do not add buy/sell recommendations without specialist regulatory and product review.

9. Known Limits and Next Work

The project is complete as a local, working prototype, but a responsible next version needs evaluation and product hardening.

- Forecast quality has not yet been benchmarked on a held-out Indian-market test set. A backtesting module should compare forecasts with actual future values across many tickers and time windows.
- yfinance is appropriate for prototyping but not a production market-data contract. Exchange licensing, data delay, and coverage must be resolved for deployment.
- The model currently uses fixed lookback and forecast lengths. Future UI work can make them controlled options while preserving model-context validation.
- The dashboard can now be beautified after the workflow is stable: stronger visual hierarchy, charts, loading states, error recovery, and mobile polish are the right next improvements.

10. Primary Sources

These official sources informed the technical choices in this guide.

Kronos official GitHub repository and model information: <https://github.com/shiyu-coder/Kronos>

OpenAI GPT-5 mini model documentation: <https://developers.openai.com/api/docs/models/gpt-5-mini>

OpenAI API authentication guidance: <https://developers.openai.com/api/reference/overview#authentication>

yfinance official repository: <https://github.com/ranaroussi/yfinance>

Apple Design Skills UI reference supplied for the dashboard: <https://github.com/s1gmamale1/apple-design-skills>

Apple Bento Grid UI reference supplied for the dashboard: <https://github.com/hubeiqiao/apple-bento-grid>